Learning C Programming with Worked Examples

Zohaib Hasan

April 12, 2026

Contents

1 Introduction to C Programming **13**
1.1 Why Learn C? 13
1.2 How a C Program Looks 13
1.2.1 First Program: Hello, World 14
1.3 Compiling and Running Your Program 14
1.3.1 Using GCC on Linux or macOS 14
1.3.2 Using GCC (MinGW) on Windows 15
1.4 Basic Building Blocks: Variables and Data Types 15
1.4.1 Common Data Types 15
1.4.2 Example: Declaring and Printing Variables 15
1.5 Taking Input from the User 16
1.5.1 Reading an Integer 16
1.5.2 Reading Multiple Values 17
1.6 Your First Mini Exercise 17

2 Data Types and Operators **19**
2.1 What Are Data Types? 19
2.1.1 Basic Built-in Data Types 19
2.1.2 Example: Using `char` and `double` 19
2.2 Constants and Variables 20
2.2.1 Example: Constants and Variables 20
2.3 Arithmetic Operators 21
2.3.1 List of Arithmetic Operators 21
2.3.2 Example: Basic Arithmetic 21

2.4 Relational and Logical Operators 22
2.4.1 Relational Operators 22
2.4.2 Logical Operators 23
2.4.3 Example: Using Relational and Logical Operators . 23
2.5 Increment and Decrement Operators 23
2.5.1 Example: Pre and Post Increment 24
2.6 Bitwise Operators 24
2.6.1 Common Bitwise Operators 24
2.6.2 Example: Bitwise Demo 25
2.7 Operator Precedence (Brief Idea) 26
2.7.1 Example: Using Parentheses 26
2.8 Practice Exercise 27

3 Control Flow: Decisions and Loops 29
3.1 Introduction 29
3.2 The `if` Statement 29
3.2.1 Syntax 29
3.2.2 Example: Check Even or Odd 30
3.3 Nested `if-else` 30
3.3.1 Example: Positive, Negative, or Zero 31
3.4 The `switch` Statement 31
3.4.1 Syntax 31
3.4.2 Example: Day of the Week 32
3.5 The `for` Loop 33
3.5.1 Syntax 33
3.5.2 Example: Print Numbers from 1 to 5 33
3.6 The `while` Loop 34
3.6.1 Syntax 34
3.6.2 Example: Count Down from 5 to 1 34
3.7 The `do-while` Loop 35
3.7.1 Syntax 35
3.7.2 Example: Simple Menu 35
3.8 `break` and `continue` 36

3.8.1 `break` 36
3.8.2 Example: Stop When Number is Negative 36
3.8.3 `continue` 37
3.8.4 Example: Skip Negative Numbers 37
3.9 Common Mistakes to Avoid 38
3.10 Practice Exercise 38

4 Functions and Recursion **39**
4.1 Why Use Functions? 39
4.2 Basics of Functions 39
4.2.1 Example: Function to Add Two Numbers 40
4.3 Function Declaration and Definition 40
4.3.1 Example: Using a Function Prototype 41
4.4 Pass by Value 41
4.4.1 Example: Demonstrating Pass by Value 41
4.5 Introduction to Recursion 42
4.6 Factorial Using Recursion 43
4.6.1 Example: Recursive Factorial 43
4.7 Fibonacci Numbers Using Recursion 44
4.7.1 Example: Recursive Fibonacci 44
4.8 Sum of Digits Using Recursion 45
4.8.1 Idea 45
4.8.2 Example: Sum of Digits 46
4.9 Power Function Using Recursion 47
4.9.1 Example: Recursive Power 47
4.10 When to Avoid Recursion 48
4.11 Practice Exercises 48

5 Arrays and Strings **49**
5.1 Why We Need Arrays 49
5.2 One-Dimensional Arrays 49
5.2.1 Declaring and Initializing Arrays 49
5.2.2 Accessing Elements 50

5.3 Reading and Printing Array Elements 50
5.3.1 Example: Read and Print 5 Integers 50
5.4 Finding Sum and Average of an Array 51
5.4.1 Example: Sum and Average of Marks 51
5.5 Finding Maximum Element in an Array 52
5.5.1 Idea 52
5.5.2 Example: Maximum of 5 Numbers 52
5.6 Linear Search in an Array 53
5.6.1 Idea 53
5.6.2 Example: Linear Search 53
5.7 Two-Dimensional Arrays (Matrices) 54
5.7.1 Declaring and Using a 2D Array 54
5.8 Matrix Addition 55
5.8.1 Example: Add Two 2x2 Matrices 55
5.9 Introduction to Strings 56
5.9.1 Declaring Strings 56
5.10 Reading and Printing Strings 56
5.10.1 Using `scanf` and `printf` 56
5.10.2 Using `gets` and `puts` (Not Recommended in New Code) 57
5.11 Simple String Length (Manually) 58
5.11.1 Example: Manual `strlen` 58
5.12 Reversing a String 58
5.12.1 Idea 58
5.12.2 Example: Reverse a String 59
5.13 Common Mistakes with Arrays and Strings 60
5.14 Practice Exercises 60

6 Pointers **61**
6.1 What is a Pointer? 61
6.2 Addresses and the `&` Operator 61
6.2.1 Example: Printing the Address of a Variable 61
6.3 Pointer Variables and the `*` Operator 62

6.3.1 Declaring a Pointer 62
6.3.2 Example: Basic Pointer Usage 62
6.4 Changing Values Through Pointers 63
6.4.1 Example: Modify a Variable Using a Pointer 63
6.5 Pointers and Arrays 64
6.5.1 Example: Array Name as Pointer 64
6.6 Pointer Arithmetic 65
6.6.1 Example: Traversing an Array Using a Pointer . . . 65
6.7 Pointers and Functions 66
6.7.1 Example: Swap Two Numbers Using Pointers . . . 66
6.8 Pointers and Strings 67
6.8.1 Example: Printing a String Using a Pointer 67
6.9 Pointer to Pointer 68
6.9.1 Example: Pointer to Pointer 68
6.10 Null Pointer 68
6.10.1 Example: Using NULL 68
6.11 Common Mistakes with Pointers 69
6.12 Practice Exercises 69

7 Structures **71**
7.1 Why We Need Structures 71
7.2 Defining a Structure 71
7.2.1 Example: Student Record 72
7.3 Using `typedef` with Structures 73
7.3.1 Example: Using `typedef` 73
7.4 Arrays of Structures 74
7.4.1 Example: Array of Books 74
7.5 Structures and Functions 77
7.6 Example: Complex Number Operations 77
7.6.1 Example: Complex Numbers 77
7.7 Example: Distance Between Two Points 79
7.7.1 Example: Coordinate Distance 79
7.8 Example: Adding Two Times 80

7.8.1 Example: Time Addition 80
7.9 Common Mistakes with Structures 82
7.10 Practice Exercises 82

8 Pattern Printing **85**
8.1 Why Pattern Printing Matters 85
8.2 The Basic Idea: Nested Loops 85
8.3 Pattern 1: Right-Angled Star Triangle 85
8.3.1 Output (for 5 rows) 86
8.3.2 Code 86
8.4 Pattern 2: Inverted Right-Angled Star Triangle 86
8.4.1 Output (for 5 rows) 87
8.4.2 Code 87
8.5 Pattern 3: Number Pyramid with Spaces 88
8.5.1 Output (for 5 rows) 88
8.5.2 Code 88
8.6 Pattern 4: Powers of 2 Pyramid 89
8.6.1 Output (for 5 rows) 89
8.6.2 Code 89
8.7 Pattern 5: Number Diamond 90
8.7.1 Output (for 5 rows upper half) 90
8.7.2 Code 91
8.8 Pattern 6: Hollow Number Triangle 91
8.8.1 Output (for 5 rows) 92
8.8.2 Code 92
8.9 Pattern 7: Floyd's Triangle 93
8.9.1 Output (for 5 rows) 93
8.9.2 Code 93
8.10 Pattern 8: Pascal's Triangle 94
8.10.1 Output (for 5 rows) 94
8.10.2 Code 94
8.11 Pattern 9: Hollow Diamond 95
8.11.1 Output (for 5 rows) 95

8.11.2 Code 95
8.12 Pattern 10: Square of Stars 97
8.12.1 Output (for 5 rows) 97
8.12.2 Code 97
8.13 Pattern 11: Chessboard Pattern 98
8.13.1 Output 98
8.13.2 Code 98
8.14 Practice Exercises 100

9 Sorting Algorithms 101
9.1 Introduction to Sorting 101
9.2 Bubble Sort 102
9.2.1 Concept 102
9.2.2 Algorithm 102
9.2.3 Time Complexity 102
9.2.4 Space Complexity 102
9.2.5 Example Code 103
9.2.6 Output 103
9.3 Insertion Sort 104
9.3.1 Concept 104
9.3.2 Algorithm 104
9.3.3 Time Complexity 104
9.3.4 Space Complexity 104
9.3.5 Example Code 105
9.3.6 Output 105
9.4 Selection Sort 105
9.4.1 Concept 105
9.4.2 Algorithm 106
9.4.3 Time Complexity 106
9.4.4 Space Complexity 106
9.4.5 Example Code 106
9.4.6 Output 107
9.5 Quick Sort 107

9.5.1 Concept . . . 107
9.5.2 Algorithm . . . 107
9.5.3 Time Complexity . . . 108
9.5.4 Space Complexity . . . 108
9.5.5 Example Code . . . 108
9.6 Shell Sort . . . 110
9.6.1 Concept . . . 110
9.6.2 Algorithm . . . 110
9.6.3 Time Complexity . . . 110
9.6.4 Space Complexity . . . 110
9.6.5 Example Code . . . 111
9.7 Comparison of Sorting Algorithms . . . 112
9.8 Summary . . . 112

10 Miscellaneous Programs and Projects 115
10.1 Introduction . . . 115
10.2 Book My Show – Cinema Seat Booking System . . . 116
10.2.1 Overview . . . 116
10.2.2 Features . . . 116
10.2.3 Data Structures Used . . . 116
10.2.4 Code . . . 117
10.3 Decimal to Any Base Conversion . . . 121
10.3.1 Overview . . . 121
10.3.2 Algorithm . . . 122
10.3.3 Code . . . 122
10.4 Square Root Calculator . . . 124
10.4.1 Overview . . . 124
10.4.2 Algorithm . . . 124
10.4.3 Code . . . 125
10.5 Fahrenheit to Celsius Converter . . . 127
10.5.1 Overview . . . 127
10.5.2 Code . . . 127
10.6 Leap Year Checker . . . 128

10.6.1 Overview 128
10.6.2 Rules for Leap Year 128
10.6.3 Code 128
10.7 Greatest Common Divisor (GCD) 130
10.7.1 Overview 130
10.7.2 Algorithm 130
10.7.3 Code 130
10.8 Check Prime Number 132
10.8.1 Overview 132
10.8.2 Code 132
10.9 Stack Implementation 134
10.9.1 Overview 134
10.9.2 Operations Supported 134
10.9.3 Code 134

Chapter 1

Introduction to C Programming

1.1 Why Learn C?

C is a powerful general-purpose programming language that is widely used for system software, embedded systems, and performance-critical applications. It helps you understand how a computer manages memory, performs calculations, and executes instructions step by step. Many modern languages like C++, Java, and C# are influenced by C, so learning C makes it easier to learn other languages later.

1.2 How a C Program Looks

Every C program follows a basic pattern:

- You include header files using `#include`.
- You define a `main` function, which is where the program starts running.
- Inside `main`, you write statements that do the actual work.

1.2.1 First Program: Hello, World

```c
#include <stdio.h>

int main() {
    printf("Hello, World!\n");
    return 0;
}
```

Line by Line Explanation

- `#include <stdio.h>` tells the compiler to include the Standard Input/Output library so we can use `printf`.
- `int main()` defines the main function that returns an integer to the operating system.
- `printf("Hello, World!\n");` prints text on the screen; `\n` moves the cursor to the next line.
- `return 0;` indicates that the program finished successfully.

1.3 Compiling and Running Your Program

Once you have written your C code in a file (for example `hello.c`), you need to compile it to create an executable program.

1.3.1 Using GCC on Linux or macOS

Open a terminal in the folder that contains `hello.c` and type:

```
gcc hello.c -o hello
./hello
```

1.3.2 Using GCC (MinGW) on Windows

Open `cmd` or PowerShell in the folder with `hello.c` and type:

```
gcc hello.c -o hello.exe
hello.exe
```

If the compilation succeeds, you will see the output:

```
Hello, World!
```

1.4 Basic Building Blocks: Variables and Data Types

A *variable* is a named location in memory where you can store a value. In C, every variable has a data type, which tells the compiler what kind of value will be stored.

1.4.1 Common Data Types

- `int` for integer values (for example `10`, `-5`).
- `float` and `double` for real numbers with decimal points (for example `3.14`).
- `char` for a single character (for example `'A'`, `'0'`).

1.4.2 Example: Declaring and Printing Variables

```c
#include <stdio.h>

int main() {
    int age = 20;
    float height = 5.8;
    char grade = 'A';

    printf("Age = %d\n", age);
    printf("Height = %.1f\n", height);
```

```
    printf("Grade = %c\n", grade);

    return 0;
}
```

Explanation

- `int age = 20;` creates an integer variable named `age` and stores 20 in it.
- `float height = 5.8;` stores a decimal value in `height`.
- `char grade = 'A';` stores the character `'A'` in `grade`.
- In `printf`, `%d` prints an integer, `%f` prints a floating-point number, and `%c` prints a character.

1.5 Taking Input from the User

To make your program interactive, you can read values from the user using `scanf`.

1.5.1 Reading an Integer

```
#include <stdio.h>

int main() {
    int number;

    printf("Enter an integer: ");
    scanf("%d", &number);

    printf("You entered %d\n", number);

    return 0;
}
```

Explanation

- `int number;` declares a variable to hold the user input.
- `scanf("%d", &number);` reads an integer from the keyboard and stores it in `number`.
- The `&` symbol gives the address of the variable to `scanf`, which is required.

1.5.2 Reading Multiple Values

```c
#include <stdio.h>

int main() {
    int a, b;

    printf("Enter two integers: ");
    scanf("%d %d", &a, &b);

    printf("Sum = %d\n", a + b);

    return 0;
}
```

1.6 Your First Mini Exercise

Try to write and run a program that:

- Asks the user for their name (one word) and age.
- Prints a message like: `Hello Alice, you are 15 years old.`

You can use `%s` in `scanf` and `printf` to handle a simple word as a string.

In the next chapters, we will build on these basics and explore operators, decision making, loops, arrays, and many example programs from

your practice set. This book will use small, focused examples from your collection to illustrate each new idea step by step. [page:1]

Chapter 2

Data Types and Operators

2.1 What Are Data Types?

In C, every variable must have a data type that tells the compiler what kind of value it will store. This helps the compiler decide how much memory to reserve and how to interpret the bits stored in that memory.

2.1.1 Basic Built-in Data Types

Some commonly used data types in C are:

- `int` - for integer (whole) numbers like `10`, `-5`, `0`.
- `float` - for real numbers with decimal points like `3.14`.
- `double` - for real numbers with more precision than `float`.
- `char` - for single characters like `'A'`, `'z'`, `'0'`.

2.1.2 Example: Using `char` and `double`

```c
#include <stdio.h>

int main() {
    char grade = 'A';
```

```
    double pi = 3.14159;

    printf("Grade = %c\n", grade);
    printf("Value of pi = %lf\n", pi);

    return 0;
}
```

Explanation

- `char grade = 'A';` stores the character `'A'` in the variable `grade`.
- `double pi = 3.14159;` stores a real number with decimal places in `pi`.
- In `printf`, `%c` prints a character and `%lf` prints a `double` value.

2.2 Constants and Variables

A *variable* is a named location in memory whose value can change during program execution. A *constant* is a value that does not change.

2.2.1 Example: Constants and Variables

```
#include <stdio.h>

int main() {
    const double PI = 3.14159;
    int radius = 5;

    double area = PI * radius * radius;

    printf("Radius = %d\n", radius);
    printf("Area of circle = %lf\n", area);

    return 0;
```

```
}
```

Explanation

- `const double PI = 3.14159;` declares a constant named `PI`.
- `radius` is a variable; you can change its value later if needed.
- The expression `PI * radius * radius` calculates the area of a circle.

2.3 Arithmetic Operators

Arithmetic operators are used to perform basic mathematical operations on numeric data types.

2.3.1 List of Arithmetic Operators

- `+` addition
- `-` subtraction
- `*` multiplication
- `/` division
- `%` remainder (modulus) for integers

2.3.2 Example: Basic Arithmetic

```c
#include <stdio.h>

int main() {
    int a = 10, b = 3;

    printf("a + b = %d\n", a + b);
    printf("a - b = %d\n", a - b);
    printf("a * b = %d\n", a * b);
```

```
9       printf("a / b = %d\n", a / b);
10      printf("a %% b = %d\n", a % b);
11
12      return 0;
13  }
```

Dry Run

```
a = 10, b = 3


a + b = 13
a - b = 7
a * b = 30
a / b = 3   (integer division, decimal part is discarded)
a % b = 1   (remainder when 10 is divided by 3)
```

2.4 Relational and Logical Operators

Relational and logical operators are used to compare values and combine conditions. These operators are very important in decision making (if-else, loops).

2.4.1 Relational Operators

- == equal to
- != not equal to
- > greater than
- < less than
- >= greater than or equal to
- <= less than or equal to

2.4.2 Logical Operators

- `&&` logical AND
- `||` logical OR
- `!` logical NOT

2.4.3 Example: Using Relational and Logical Operators

```c
#include <stdio.h>

int main() {
    int age = 16;
    int hasID = 1;  // 1 means yes, 0 means no

    if (age >= 18 && hasID == 1) {
        printf("Allowed to enter.\n");
    } else {
        printf("Not allowed to enter.\n");
    }

    return 0;
}
```

2.5 Increment and Decrement Operators

These operators are shortcuts to add or subtract 1 from a variable.

- `++` increment operator
- `--` decrement operator

They can appear before or after the variable:

- `++a` pre-increment (increase first, then use the value)
- `a++` post-increment (use the value, then increase)

2.5.1 Example: Pre and Post Increment

```c
#include <stdio.h>

int main() {
    int a = 5, b, c;

    b = ++a;  // a becomes 6, then b gets 6
    printf("After b = ++a; a = %d, b = %d\n", a, b);

    c = a++;  // c gets 6, then a becomes 7
    printf("After c = a++; a = %d, c = %d\n", a, c);

    return 0;
}
```

2.6 Bitwise Operators

Bitwise operators work on the binary representation of integers. They are useful for low-level programming, such as working with flags, bits, and hardware. [page:1]

2.6.1 Common Bitwise Operators

- & bitwise AND
- | bitwise OR
- ^ bitwise XOR (exclusive OR)
- ~ bitwise NOT
- « left shift
- » right shift

2.6.2 Example: Bitwise Demo

```c
#include <stdio.h>

int main() {
    int a = 5;   // 0101 in binary (in 4 bits)
    int b = 3;   // 0011 in binary

    printf("a & b = %d\n", a & b);   // 0001 = 1
    printf("a | b = %d\n", a | b);   // 0111 = 7
    printf("a ^ b = %d\n", a ^ b);   // 0110 = 6

    printf("a << 1 = %d\n", a << 1); // 1010 = 10
    printf("a >> 1 = %d\n", a >> 1); // 0010 = 2

    return 0;
}
```

Dry Run (4-bit representation)

```
a = 5  -> 0101
b = 3  -> 0011

a & b  -> 0001 = 1   (both bits must be 1)
a | b  -> 0111 = 7   (at least one bit is 1)
a ^ b  -> 0110 = 6   (bits are different)

a << 1 -> 1010 = 10  (shift left, multiply by 2)
a >> 1 -> 0010 = 2   (shift right, divide by 2)
```

2.7 Operator Precedence (Brief Idea)

When multiple operators appear in the same expression, C follows a fixed priority order called *operator precedence*. For beginners, it is safer

to use parentheses `( )` to clearly show the order in which you want operations to happen.

2.7.1 Example: Using Parentheses

```c
#include <stdio.h>

int main() {
    int result1 = 10 + 5 * 2;
    int result2 = (10 + 5) * 2;

    printf("result1 = %d\n", result1); // 20
    printf("result2 = %d\n", result2); // 30

    return 0;
}
```

Explanation

- In `result1`, multiplication has higher precedence, so `5 * 2` is done first, then 10 is added.
- In `result2`, parentheses force `10 + 5` to be done first, then the result is multiplied by 2.

2.8 Practice Exercise

Write a program that:

- Asks the user to enter two integers.
- Prints their sum, difference, product, quotient, and remainder.
- Also prints whether the first number is greater than, less than, or equal to the second number.

Try to use arithmetic, relational, and logical operators in your solution.

Chapter 3

Control Flow: Decisions and Loops

3.1 Introduction

So far you have seen how to declare variables, read input, and perform calculations. In real programs, we also need to make decisions and repeat actions. In C, decisions are made using `if`, `if-else`, and `switch`, and repetition is done using loops like `for`, `while`, and `do-while`.

3.2 The `if` Statement

The `if` statement lets your program choose whether to execute a block of code based on a condition.

3.2.1 Syntax

```
1 if (condition) {
2     /* statements if condition is true */
3 }
```

If the condition is non-zero (true), the statements inside the braces run; otherwise they are skipped.

3.2.2 Example: Check Even or Odd

```c
#include <stdio.h>

int main() {
    int number;

    printf("Enter an integer: ");
    scanf("%d", &number);

    if (number % 2 == 0) {
        printf("Even number\n");
    } else {
        printf("Odd number\n");
    }

    return 0;
}
```

Explanation

- `number % 2` gives the remainder when `number` is divided by 2.
- If the remainder is 0, the number is even; otherwise, it is odd.
- The `if` and `else` blocks contain different messages for each case.

3.3 Nested `if-else`

Sometimes you need to check more than one condition in sequence. You can place one `if-else` inside another; this is called nesting.

3.3.1 Example: Positive, Negative, or Zero

```c
#include <stdio.h>

int main() {
```

```c
    int number;

    printf("Enter an integer: ");
    scanf("%d", &number);

    if (number > 0) {
        printf("Positive\n");
    } else if (number < 0) {
        printf("Negative\n");
    } else {
        printf("Zero\n");
    }

    return 0;
}
```

3.4 The `switch` Statement

The `switch` statement is useful when you want to choose from many options based on the value of a single expression (usually an `int` or `char`).

3.4.1 Syntax

```c
switch (expression) {
    case value1:
        /* statements */
        break;
    case value2:
        /* statements */
        break;
    ...
    default:
        /* statements if no case matches */
}
```

3.4.2 Example: Day of the Week

```c
#include <stdio.h>

int main() {
    int day;

    printf("Enter day number (1-3): ");
    scanf("%d", &day);

    switch (day) {
        case 1:
            printf("Monday\n");
            break;
        case 2:
            printf("Tuesday\n");
            break;
        case 3:
            printf("Wednesday\n");
            break;
        default:
            printf("Invalid day\n");
    }

    return 0;
}
```

Explanation

- The value of `day` is compared with each `case` label.
- `break;` stops execution from falling through to the next case.
- `default` runs if none of the cases match.

3.5 The for Loop

A `for` loop is commonly used when you know in advance how many times you want to repeat something.

3.5.1 Syntax

```
for (initialization; condition; update) {
    /* statements to repeat */
}
```

3.5.2 Example: Print Numbers from 1 to 5

```
#include <stdio.h>

int main() {
    int i;

    for (i = 1; i <= 5; i++) {
        printf("%d ", i);
    }

    return 0;
}
```

Dry Run

```
Start: i = 1
Check: i <= 5? Yes -> print 1, then i becomes 2
Check: i <= 5? Yes -> print 2, then i becomes 3
Check: i <= 5? Yes -> print 3, then i becomes 4
Check: i <= 5? Yes -> print 4, then i becomes 5
Check: i <= 5? Yes -> print 5, then i becomes 6
Check: i <= 5? No  -> loop stops
```

3.6 The `while` Loop

A `while` loop repeats a block of code as long as a condition remains true. It is useful when you do not know in advance how many times you will loop.

3.6.1 Syntax

```
while (condition) {
    /* statements */
}
```

3.6.2 Example: Count Down from 5 to 1

```
#include <stdio.h>

int main() {
    int i = 5;

    while (i >= 1) {
        printf("%d ", i);
        i--;
    }

    return 0;
}
```

3.7 The `do-while` Loop

A `do-while` loop is similar to a `while` loop, but it guarantees that the body runs at least once because the condition is checked at the end.

3.7.1 Syntax

```
do {
```

```
    /* statements */
} while (condition);
```

3.7.2 Example: Simple Menu

```c
#include <stdio.h>

int main() {
    int choice;

    do {
        printf("1. Say Hello\n");
        printf("2. Say Bye\n");
        printf("3. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        if (choice == 1) {
            printf("Hello!\n");
        } else if (choice == 2) {
            printf("Bye!\n");
        } else if (choice != 3) {
            printf("Invalid choice\n");
        }

    } while (choice != 3);

    return 0;
}
```

3.8 break and continue

3.8.1 break

break is used inside loops or switch statements to stop execution immediately and jump out of the enclosing structure.

3.8.2 Example: Stop When Number is Negative

```c
#include <stdio.h>

int main() {
    int i, x;

    for (i = 1; i <= 5; i++) {
        printf("Enter a number: ");
        scanf("%d", &x);

        if (x < 0) {
            printf("Negative number entered. Stopping.\n");
            break;
        }

        printf("You entered %d\n", x);
    }

    return 0;
}
```

3.8.3 continue

continue skips the rest of the current loop body and jumps to the next iteration.

3.8.4 Example: Skip Negative Numbers

```c
#include <stdio.h>
```

```c

int main() {
    int i, x;

    for (i = 1; i <= 5; i++) {
        printf("Enter a number: ");
        scanf("%d", &x);

        if (x < 0) {
            printf("Skipping negative number.\n");
            continue;
        }

        printf("You entered %d\n", x);
    }

    return 0;
}
```

3.9 Common Mistakes to Avoid

- Forgetting braces `{}` around multiple statements in an `if` or loop.
- Using `=` instead of `==` inside conditions (assignment vs comparison).
- Creating infinite loops by never changing the loop variable or writing a condition that is always true.

3.10 Practice Exercise

Write a program that:

- Displays a menu with options for addition, subtraction, multiplication, and exit.

- Asks the user to enter two numbers and an option.
- Performs the selected operation using `if-else` or `switch`.
- Repeats the menu until the user chooses to exit (use a `do-while` loop).

Try to combine what you learned about decisions and loops to solve this problem.

Chapter 4

Functions and Recursion

4.1 Why Use Functions?

As programs grow larger, it becomes difficult to manage all the code inside a single `main` function. Functions help you split a big program into smaller, reusable blocks that each perform one specific task. This makes your code easier to read, test, and debug.

4.2 Basics of Functions

A function in C has:

- A *return type* (what type of value it gives back, for example `int`, `float`, `void`).
- A *name* (for example `sum`, `max`).
- A list of *parameters* inside parentheses (input values for the function).
- A *body* inside braces `{}` that contains statements.

4.2.1 Example: Function to Add Two Numbers

```
1 #include <stdio.h>
```

```c

int add(int a, int b) {   // function definition
    int result = a + b;
    return result;        // return the sum to the caller
}

int main() {
    int x = 5, y = 7;
    int s;

    s = add(x, y);        // function call
    printf("Sum = %d\n", s);

    return 0;
}
```

Explanation

- `int add(int a, int b)` declares a function named `add` that takes two integers and returns an integer.
- Inside the function, we calculate `result` and `return` it.
- In `main`, we call `add(x, y)` and store the returned value in `s`.

4.3 Function Declaration and Definition

Sometimes you write the function body after `main`. In that case, you should declare the function before `main` so the compiler knows its name and parameters.

4.3.1 Example: Using a Function Prototype

```c
#include <stdio.h>

// function declaration (prototype)
```

```c
int square(int n);

int main() {
    int num = 4;
    int ans = square(num);

    printf("Square of %d is %d\n", num, ans);

    return 0;
}

// function definition
int square(int n) {
    return n * n;
}
```

4.4 Pass by Value

In C, function arguments are passed by value. This means the function gets a copy of the variable, so changing the parameter inside the function does not affect the original variable in `main`.

4.4.1 Example: Demonstrating Pass by Value

```c
#include <stdio.h>

void change(int x) {
    x = 100;
    printf("Inside change, x = %d\n", x);
}

int main() {
    int a = 10;

    printf("Before change, a = %d\n", a);
    change(a);
```

```
    printf("After change, a = %d\n", a);

    return 0;
}
```

Output (Expected)

```
Before change, a = 10
Inside change, x = 100
After change, a = 10
```

Explanation

- Inside `change`, `x` is a copy of `a`.
- Changing `x` does not change `a` in `main`.

4.5 Introduction to Recursion

A function is called *recursive* if it calls itself. Recursion is useful for solving problems that can be broken into smaller subproblems of the same type, such as factorial, Fibonacci numbers, and sum of digits. [page:1]

Every recursive function must have:

- A *base case* - a simple situation where the function does not call itself.
- A *recursive case* - where the function calls itself with a smaller or simpler input.

4.6 Factorial Using Recursion

The factorial of a positive integer n is defined as:

$$n! = n \times (n-1) \times (n-2) \times \cdots \times 1$$

By definition, $0! = 1$.

4.6.1 Example: Recursive Factorial

```c
#include <stdio.h>

int factorial(int n) {
    if (n == 0 || n == 1) {   // base case
        return 1;
    } else {                  // recursive case
        return n * factorial(n - 1);
    }
}

int main() {
    int num;

    printf("Enter a non-negative integer: ");
    scanf("%d", &num);

    if (num < 0) {
        printf("Factorial is not defined for negative numbers.\n");
    } else {
        printf("Factorial of %d is %d\n", num, factorial(num));
    }

    return 0;
}
```

Dry Run for `factorial(3)`

```
factorial(3)
= 3 * factorial(2)
= 3 * (2 * factorial(1))
= 3 * (2 * 1)   // factorial(1) returns 1 (base case)
= 3 * 2
```

```
= 6
```

4.7 Fibonacci Numbers Using Recursion

The Fibonacci sequence is a series where each number is the sum of the previous two numbers:

$$0,\ 1,\ 1,\ 2,\ 3,\ 5,\ 8,\ 13,\ \ldots$$

The definitions are:

$$F(0) = 0, \quad F(1) = 1, \quad F(n) = F(n-1) + F(n-2) \text{ for } n \geq 2$$

4.7.1 Example: Recursive Fibonacci

```c
#include <stdio.h>

int fib(int n) {
    if (n == 0) {        // base case 1
        return 0;
    } else if (n == 1) { // base case 2
        return 1;
    } else {             // recursive case
        return fib(n - 1) + fib(n - 2);
    }
}

int main() {
    int n, i;

    printf("Enter how many terms: ");
    scanf("%d", &n);

    printf("Fibonacci sequence:\n");
    for (i = 0; i < n; i++) {
```

```
        printf("%d ", fib(i));
    }

    return 0;
}
```

4.8 Sum of Digits Using Recursion

We can also use recursion to add the digits of a number. For example, the sum of digits of 1234 is $1 + 2 + 3 + 4 = 10$.

4.8.1 Idea

- Base case: if the number is 0, the sum is 0.
- Recursive case: sum of digits of `n` is `last digit + sum of digits of remaining number`.

4.8.2 Example: Sum of Digits

```c
#include <stdio.h>

int sumOfDigits(int n) {
    if (n == 0) {                // base case
        return 0;
    } else {                     // recursive case
        return (n % 10) + sumOfDigits(n / 10);
    }
}

int main() {
    int num;

    printf("Enter a number: ");
    scanf("%d", &num);

```

```c
    if (num < 0) {
        num = -num;  // handle negative numbers
    }

    printf("Sum of digits = %d\n", sumOfDigits(num));

    return 0;
}
```

Dry Run for `sumOfDigits(123)`

```
sumOfDigits(123)
= 3 + sumOfDigits(12)
= 3 + (2 + sumOfDigits(1))
= 3 + (2 + (1 + sumOfDigits(0)))
= 3 + (2 + (1 + 0))
= 3 + (2 + 1)
= 3 + 3
= 6
```

4.9 Power Function Using Recursion

We can compute a^b (a raised to power b) recursively:

$$a^0 = 1, \quad a^b = a \times a^{b-1} \text{ for } b > 0$$

4.9.1 Example: Recursive Power

```c
#include <stdio.h>

int power(int base, int exp) {
    if (exp == 0) {          // base case
        return 1;
    } else {                 // recursive case
        return base * power(base, exp - 1);
```

```
    }
}

int main() {
    int a, b;

    printf("Enter base and exponent: ");
    scanf("%d %d", &a, &b);

    if (b < 0) {
        printf("This simple version does not handle negative
        ↪  exponents.\n");
    } else {
        printf("%d^%d = %d\n", a, b, power(a, b));
    }

    return 0;
}
```

4.10 When to Avoid Recursion

Recursion is elegant, but it is not always the best choice. For large inputs, some recursive solutions (like the simple Fibonacci function) can be slow and use a lot of memory. In such cases, it is better to use an iterative approach with loops.

4.11 Practice Exercises

Try to write the following recursive functions:

- A function that finds the greatest common divisor (GCD) of two numbers using Euclid's algorithm.
- A function that reverses a number (for example, 1234 becomes 4321).

- A function that prints numbers from 1 to `n` using recursion.

In each case, clearly identify the base case and the recursive case before writing the code.

Chapter 5

Arrays and Strings

5.1 Why We Need Arrays

Sometimes we need to store many values of the same type, such as the marks of 50 students or the prices of 100 items. Using 50 separate variables like `mark1`, `mark2`, ..., `mark50` is difficult and confusing. An *array* lets us store many values of the same type in a single collection, using an index to access each element.

5.2 One-Dimensional Arrays

A one-dimensional (1D) array is like a row of boxes, each box holding one value of the same type.

5.2.1 Declaring and Initializing Arrays

```
int marks[5];                 // declare an array of 5 integers
int numbers[4] = {10, 20, 30, 40}; // declare and initialize
```

5.2.2 Accessing Elements

Array indices in C start from 0. So, for `int numbers[4] = {10,20,30,40};`:

- `numbers[0]` is 10.
- `numbers[1]` is 20.
- `numbers[2]` is 30.
- `numbers[3]` is 40.

5.3 Reading and Printing Array Elements

5.3.1 Example: Read and Print 5 Integers

```c
#include <stdio.h>

int main() {
    int a[5];
    int i;

    printf("Enter 5 integers:\n");
    for (i = 0; i < 5; i++) {
        scanf("%d", &a[i]);
    }

    printf("You entered:\n");
    for (i = 0; i < 5; i++) {
        printf("%d ", a[i]);
    }

    return 0;
}
```

Explanation

- The first `for` loop reads values into `a[0]` to `a[4]`.
- The second `for` loop prints each element in order.

5.4 Finding Sum and Average of an Array

5.4.1 Example: Sum and Average of Marks

```c
#include <stdio.h>

int main() {
    int marks[5];
    int i, sum = 0;
    float average;

    printf("Enter marks of 5 students:\n");
    for (i = 0; i < 5; i++) {
        scanf("%d", &marks[i]);
        sum = sum + marks[i];
    }

    average = sum / 5.0;

    printf("Sum = %d\n", sum);
    printf("Average = %.2f\n", average);

    return 0;
}
```

5.5 Finding Maximum Element in an Array

5.5.1 Idea

- Assume the first element is the maximum.
- Compare it with every other element.
- If you find a bigger element, update the maximum.

5.5.2 Example: Maximum of 5 Numbers

```c
#include <stdio.h>

int main() {
    int a[5];
    int i, max;

    printf("Enter 5 integers:\n");
    for (i = 0; i < 5; i++) {
        scanf("%d", &a[i]);
    }

    max = a[0];
    for (i = 1; i < 5; i++) {
        if (a[i] > max) {
            max = a[i];
        }
    }

    printf("Maximum value = %d\n", max);

    return 0;
}
```

5.6 Linear Search in an Array

5.6.1 Idea

- Go through the array from the first element to the last.
- Compare each element with the key we are searching for.
- If a match is found, report the position and stop.

5.6.2 Example: Linear Search

```c
#include <stdio.h>

int main() {
    int a[5];
    int i, key, found = 0;

    printf("Enter 5 integers:\n");
    for (i = 0; i < 5; i++) {
        scanf("%d", &a[i]);
    }

    printf("Enter value to search: ");
    scanf("%d", &key);

    for (i = 0; i < 5; i++) {
        if (a[i] == key) {
            printf("Found at position %d\n", i + 1);
            found = 1;
            break;
        }
    }

    if (!found) {
        printf("Not found in the array.\n");
    }

    return 0;
}
```

5.7 Two-Dimensional Arrays (Matrices)

A two-dimensional (2D) array can be imagined as a table with rows and columns. For example, a 3x3 matrix of integers can be stored as `int a[3][3];`.

5.7.1 Declaring and Using a 2D Array

```c
#include <stdio.h>

int main() {
    int a[3][3];
    int i, j;

    printf("Enter elements of a 3x3 matrix:\n");
    for (i = 0; i < 3; i++) {
        for (j = 0; j < 3; j++) {
            scanf("%d", &a[i][j]);
        }
    }

    printf("Matrix is:\n");
    for (i = 0; i < 3; i++) {
        for (j = 0; j < 3; j++) {
            printf("%d ", a[i][j]);
        }
        printf("\n");
    }

    return 0;
}
```

5.8 Matrix Addition

5.8.1 Example: Add Two 2x2 Matrices

```c
#include <stdio.h>

int main() {
    int a[2][2], b[2][2], c[2][2];
    int i, j;

```

```
    printf("Enter elements of matrix A (2x2):\n");
    for (i = 0; i < 2; i++) {
        for (j = 0; j < 2; j++) {
            scanf("%d", &a[i][j]);
        }
    }

    printf("Enter elements of matrix B (2x2):\n");
    for (i = 0; i < 2; i++) {
        for (j = 0; j < 2; j++) {
            scanf("%d", &b[i][j]);
        }
    }

    // Add the matrices
    for (i = 0; i < 2; i++) {
        for (j = 0; j < 2; j++) {
            c[i][j] = a[i][j] + b[i][j];
        }
    }

    printf("Result (A + B):\n");
    for (i = 0; i < 2; i++) {
        for (j = 0; j < 2; j++) {
            printf("%d ", c[i][j]);
        }
        printf("\n");
    }

    return 0;
}
```

5.9 Introduction to Strings

In C, a *string* is an array of characters ending with a special character `'0'` called the null character. For example, the string `"HELLO"` is stored as:

```
'H'  'E'  'L'  'L'  'O'  '\0'
```

5.9.1 Declaring Strings

```c
char name[20];            // can hold up to 19 characters + '\0'
char city[] = "Jabalpur"; // size decided automatically
```

5.10 Reading and Printing Strings

5.10.1 Using `scanf` and `printf`

```c
#include <stdio.h>

int main() {
    char name[20];

    printf("Enter your name (one word): ");
    scanf("%19s", name);  // read a word, avoid overflow

    printf("Hello %s\n", name);

    return 0;
}
```

5.10.2 Using `gets` and `puts` (Not Recommended in New Code)

Some older programs use `gets`, which is unsafe and has been removed from the C standard. Modern compilers may not support it; instead you can use `fgets`. For beginners, if your compiler supports `gets`, you may see examples like:

```c
#include <stdio.h>

int main() {
    char line[50];
```

```
5
6       printf("Enter a sentence:\n");
7       gets(line);   // WARNING: unsafe, may cause buffer overflow
8
9       puts("You entered:");
10      puts(line);
11
12      return 0;
13  }
```

Prefer safer functions like `fgets` in serious programs.

5.11 Simple String Length (Manually)

We can find the length of a string by counting characters until we reach ’

0’.

5.11.1 Example: Manual `strlen`

```
1   #include <stdio.h>
2
3   int stringLength(char s[]) {
4       int i = 0;
5       while (s[i] != '\0') {
6           i++;
7       }
8       return i;
9   }
10
11  int main() {
12      char word[50];
13
14      printf("Enter a word: ");
15      scanf("%49s", word);
16
```

```
    printf("Length = %d\n", stringLength(word));

    return 0;
}
```

5.12 Reversing a String

5.12.1 Idea

- Find the length of the string.
- Swap the first and last characters, then the second and second last, and so on.

5.12.2 Example: Reverse a String

```
#include <stdio.h>

int stringLength(char s[]) {
    int i = 0;
    while (s[i] != '\0') {
        i++;
    }
    return i;
}

void reverseString(char s[]) {
    int i = 0;
    int j = stringLength(s) - 1;
    char temp;

    while (i < j) {
        temp = s[i];
        s[i] = s[j];
        s[j] = temp;
        i++;
```

```
        j--;
    }
}

int main() {
    char word[50];

    printf("Enter a word: ");
    scanf("%49s", word);

    reverseString(word);

    printf("Reversed = %s\n", word);

    return 0;
}
```

5.13 Common Mistakes with Arrays and Strings

- Using indices outside the valid range (for example, `a[5]` when the array size is 5).
- Forgetting that arrays in C start at index 0.
- Not leaving space for `'\0'` when declaring a string array.
- Mixing `scanf("%s", ...)` and `gets`/`fgets` without handling leftover newline characters.

5.14 Practice Exercises

- Write a program to find the minimum element in an array of 10 integers.

- Write a program to count how many times a given number appears in an array.
- Write a program to check if a given string is a palindrome (reads the same forward and backward).

Chapter 6

Pointers

6.1 What is a Pointer?

In C, a *pointer* is a variable that stores the memory address of another variable. Instead of holding a direct value like `10` or `'A'`, a pointer holds the location where that value is stored in memory. Pointers are very powerful and are used for arrays, strings, dynamic memory, and for passing large data efficiently to functions.

6.2 Addresses and the & Operator

Every variable in a program is stored at a unique address in memory. We can find the address of a variable using the `&` (address-of) operator.

6.2.1 Example: Printing the Address of a Variable

```c
#include <stdio.h>

int main() {
    int a = 10;

    printf("Value of a = %d\n", a);
    printf("Address of a = %p\n", (void*)&a);
```

```
8
9       return 0;
10  }
```

Explanation

- `a` holds the value `10`.
- `&a` gives the address of `a` in memory.
- `%p` is used in `printf` to print an address (a pointer).

6.3 Pointer Variables and the * Operator

A pointer variable is declared using the * symbol.

6.3.1 Declaring a Pointer

```
1   int *p;     // p is a pointer to an int
2   char *cp;   // cp is a pointer to a char
```

The * in the declaration means "pointer to". The type of the pointer must match the type of the variable whose address it will store.

6.3.2 Example: Basic Pointer Usage

```
1   #include <stdio.h>
2
3   int main() {
4       int a = 10;
5       int *p;      // declare a pointer to int
6
7       p = &a;      // p now holds the address of a
8
9       printf("Value of a = %d\n", a);
10      printf("Address of a = %p\n", (void*)&a);
```

```
    printf("Value of p (address stored) = %p\n", (void*)p);
    printf("Value pointed to by p = %d\n", *p);

    return 0;
}
```

Explanation

- `p = &a;` makes `p` point to `a`.
- `p` stores the address of `a`.
- `*p` (read as "value pointed to by p") gives the value stored at that address, which is `10`.

6.4 Changing Values Through Pointers

If a pointer points to a variable, we can use `*p` on the left side of an assignment to change that variable's value.

6.4.1 Example: Modify a Variable Using a Pointer

```
#include <stdio.h>

int main() {
    int a = 10;
    int *p = &a;

    printf("Before: a = %d\n", a);

    *p = 50;   // change the value at the address stored in p

    printf("After: a = %d\n", a);

    return 0;
}
```

6.5 Pointers and Arrays

In C, the name of an array (like `a`) often behaves like a pointer to its first element (`&a[0]`). This close relationship between arrays and pointers is very important.

6.5.1 Example: Array Name as Pointer

```c
#include <stdio.h>

int main() {
    int a[5] = {10, 20, 30, 40, 50};
    int *p;

    p = a;  // same as p = &a[0];

    printf("a[0] = %d\n", a[0]);
    printf("*p   = %d\n", *p);

    printf("a[1] = %d\n", a[1]);
    printf("*(p + 1) = %d\n", *(p + 1));

    return 0;
}
```

Explanation

- `p = a;` makes `p` point to the first element of the array.
- `*p` is the same as `a[0]`.
- `*(p + 1)` is the same as `a[1]`.

6.6 Pointer Arithmetic

Because pointers store addresses, you can add or subtract integers from them. When you write `p + 1`, the pointer moves forward by the size of its base type (for an `int *`, it moves by `sizeof(int)` bytes).

6.6.1 Example: Traversing an Array Using a Pointer

```c
#include <stdio.h>

int main() {
    int a[5] = {1, 2, 3, 4, 5};
    int *p;
    int i;

    p = a;  // point to a[0]

    printf("Array elements using pointer:\n");
    for (i = 0; i < 5; i++) {
        printf("%d ", *(p + i));
    }

    return 0;
}
```

6.7 Pointers and Functions

Pointers are often used to allow a function to modify variables in the caller (because normal parameters are passed by value). By passing the address of a variable, the function can change the original variable directly.

6.7.1 Example: Swap Two Numbers Using Pointers

```c
#include <stdio.h>
```

```c

void swap(int *x, int *y) {
    int temp;

    temp = *x;
    *x = *y;
    *y = temp;
}

int main() {
    int a = 10, b = 20;

    printf("Before swap: a = %d, b = %d\n", a, b);
    swap(&a, &b);
    printf("After swap:  a = %d, b = %d\n", a, b);

    return 0;
}
```

Explanation

- `swap` takes pointers to `int` as parameters.
- `swap(&a, &b);` passes the addresses of `a` and `b`.
- Inside `swap`, `*x` and `*y` refer to `a` and `b`, so their values get exchanged.

6.8 Pointers and Strings

Strings in C are arrays of `char`, so pointers and arrays work closely together here as well.

6.8.1 Example: Printing a String Using a Pointer

```c
#include <stdio.h>
```

```c

int main() {
    char str[] = "Hello";
    char *p = str;

    printf("String using array name: %s\n", str);

    printf("Characters using pointer: ");
    while (*p != '\0') {
        printf("%c", *p);
        p++;
    }

    printf("\n");
    return 0;
}
```

Explanation

- `char *p = str;` makes `p` point to the first character.
- We move the pointer forward until we reach the null character `'\0'`.

6.9 Pointer to Pointer

Sometimes you may have a pointer that stores the address of another pointer. This is called a pointer to a pointer and is declared using `**`.

6.9.1 Example: Pointer to Pointer

```c
#include <stdio.h>

int main() {
    int a = 10;
    int *p = &a;
```

```
    int **pp = &p;

    printf("a  = %d\n", a);
    printf("*p = %d\n", *p);
    printf("**pp = %d\n", **pp);

    return 0;
}
```

6.10 Null Pointer

A *null pointer* is a pointer that does not point to any valid memory location. It is usually represented by `NULL`.

6.10.1 Example: Using NULL

```
#include <stdio.h>

int main() {
    int *p = NULL;

    if (p == NULL) {
        printf("Pointer is NULL, it does not point anywhere.\n");
    }

    return 0;
}
```

6.11 Common Mistakes with Pointers

- Using an uninitialized pointer (it may contain a garbage address).
- Dereferencing a null or invalid pointer (this can crash the program).

- Forgetting that array indices start at 0 and going out of bounds.
- Mixing pointer types (for example, using a `char *` to hold the address of an `int`) without proper casting.

6.12 Practice Exercises

- Write a program that uses a pointer to find the largest element in an integer array.
- Write a program that counts the number of vowels in a string using a pointer to `char`.
- Write a function that takes an integer array and its size, and reverses the array in place using pointers.

Chapter 7

Structures

7.1 Why We Need Structures

So far, you have used arrays to store many values of the *same* type. But in real life, we often need to store related pieces of information of *different* types together. For example, a student record may include:

- Roll number (a string or number)
- Name (a string)
- Phone number (a string)

Arrays cannot hold different types in one collection. For this, C provides *structures* (or `struct`). A structure lets you group different types of data under one name.

7.2 Defining a Structure

The keyword `struct` is used to define a new data type. Here is the general syntax:

```
1 struct StructureName {
2     type member1;
```

```
    type member2;
    type member3;
};
```

Each item inside the braces is called a *member* or *field* of the structure.

7.2.1 Example: Student Record

```c
#include <stdio.h>

struct record {
    char rollno[20];
    char name[30];
    char phone[20];
};

int main() {
    struct record student;

    printf("Enter student details:\n");

    printf("Roll No.: ");
    scanf("%s", student.rollno);

    printf("Name: ");
    scanf("%s", student.name);

    printf("Contact: ");
    scanf("%s", student.phone);

    printf("\nFollowing are the details of student:\n");
    printf("Roll No: %s\n", student.rollno);
    printf("Name: %s\n", student.name);
    printf("Contact: %s\n", student.phone);

    return 0;
```

```
}
```

Explanation

- `struct record` defines a new structure type with three members: `rollno`, `name`, and `phone`.
- `struct record student;` declares a variable named `student` of type `struct record`.
- The `.` (dot) operator is used to access members: `student.rollno`, `student.name`, etc.

7.3 Using `typedef` with Structures

Typing `struct` every time can be long. The `typedef` keyword lets you create a short name for the structure type.

7.3.1 Example: Using `typedef`

```c
#include <stdio.h>

struct record {
    char rollno[20];
    char name[30];
    char phone[20];
};

typedef struct record DATA;

int main() {
    DATA student;

    printf("Enter student details:\n");
    printf("Roll No.: ");
    scanf("%s", student.rollno);
```

```c

    printf("Name: ");
    scanf("%s", student.name);

    printf("Contact: ");
    scanf("%s", student.phone);

    printf("\nRoll No: %s\n", student.rollno);
    printf("Name: %s\n", student.name);
    printf("Contact: %s\n", student.phone);

    return 0;
}
```

Explanation

- `typedef struct record DATA;` creates a new type name `DATA` for `struct record`.
- Now we can write `DATA student;` instead of `struct record student;`.

7.4 Arrays of Structures

Just like arrays of `int` or `char`, you can have arrays of structures. This is useful when you need to store many records of the same kind.

7.4.1 Example: Array of Books

```c
#include <stdio.h>
#include <string.h>

struct book {
    char name[10];
    char author[30];
    int price;
};
```

```
9
10  void print(struct book b[], int n) {
11      int i;
12      for (i = 0; i < n; i++) {
13          printf("%s ", b[i].name);
14      }
15  }
16
17  void sort(struct book b[], int n) {
18      int i, j;
19      struct book temp;
20      for (i = 0; i < n - 1; i++) {
21          for (j = 0; j < n - i - 1; j++) {
22              if (b[j].price > b[j + 1].price) {
23                  temp = b[j];
24                  b[j] = b[j + 1];
25                  b[j + 1] = temp;
26              }
27          }
28      }
29  }
30
31  int main() {
32      struct book b[4];
33      int i;
34
35      // Initialize the books
36      strcpy(b[0].name, "A"); strcpy(b[0].author, "XY"); b[0].price = 52;
37      strcpy(b[1].name, "B"); strcpy(b[1].author, "PQ"); b[1].price = 39;
38      strcpy(b[2].name, "C"); strcpy(b[2].author, "MN"); b[2].price = 80;
39      strcpy(b[3].name, "D"); strcpy(b[3].author, "AB"); b[3].price = 72;
40
41      printf("Books before sorting: ");
42      print(b, 4);
43
44      sort(b, 4);
45
```

```
    printf("\nBooks after sorting by price: ");
    print(b, 4);

    printf("\n\nDetailed list after sorting:\n");
    for (i = 0; i < 4; i++) {
        printf("%s by %s - Rs.%d\n", b[i].name, b[i].author,
        ↪  b[i].price);
    }

    return 0;
}
```

Explanation

- `struct book b[4];` declares an array of 4 books.
- Each book has a `name`, `author`, and `price`.
- The `print` function displays all book names.
- The `sort` function uses bubble sort to arrange books by price (ascending).
- Notice that whole structures can be assigned: `temp = b[j];`

7.5 Structures and Functions

You can pass structures to functions and also return structures from functions. This makes your code more modular and easier to manage.

7.6 Example: Complex Number Operations

A complex number has two parts: a real part and an imaginary part. We can represent it using a structure and perform operations like addition, subtraction, and multiplication.

7.6.1 Example: Complex Numbers

```c
#include <stdio.h>

typedef struct complex {
    int real;
    int imag;
} CPLX;

CPLX add(CPLX x, CPLX y);
CPLX subtract(CPLX x, CPLX y);
CPLX multiply(CPLX x, CPLX y);

int main() {
    CPLX a, b, ans;

    printf("Enter the first complex number (real imag): ");
    scanf("%d%d", &a.real, &a.imag);

    printf("Enter the second complex number (real imag): ");
    scanf("%d%d", &b.real, &b.imag);

    ans = add(a, b);
    printf("Addition = %d + j%d\n", ans.real, ans.imag);

    ans = subtract(a, b);
    printf("Subtraction = %d + j%d\n", ans.real, ans.imag);

    ans = multiply(a, b);
    printf("Multiplication = %d + j%d\n", ans.real, ans.imag);

    return 0;
}

CPLX add(CPLX x, CPLX y) {
    CPLX ans;
    ans.real = x.real + y.real;
```

```
    ans.imag = x.imag + y.imag;
    return ans;
}

CPLX subtract(CPLX x, CPLX y) {
    CPLX ans;
    ans.real = x.real - y.real;
    ans.imag = x.imag - y.imag;
    return ans;
}

CPLX multiply(CPLX x, CPLX y) {
    CPLX ans;
    ans.real = (x.real * y.real) - (x.imag * y.imag);
    ans.imag = (x.real * y.imag) + (y.real * x.imag);
    return ans;
}
```

Mathematics Behind It

For two complex numbers $z_1 = a + jb$ and $z_2 = c + jd$:

- Addition: $z_1 + z_2 = (a + c) + j(b + d)$
- Subtraction: $z_1 - z_2 = (a - c) + j(b - d)$
- Multiplication: $z_1 \times z_2 = (ac - bd) + j(ad + bc)$

7.7 Example: Distance Between Two Points

We can use a structure to represent a point in 2D space and calculate the distance between two points.

7.7.1 Example: Coordinate Distance

```
#include <stdio.h>
```

```c
#include <math.h>

typedef struct coordinates {
    float x;
    float y;
} point;

float get_length(point a, point b);

int main() {
    point a, b;

    printf("Enter the x and y coordinates of the first point: ");
    scanf("%f%f", &a.x, &a.y);

    printf("Enter the x and y coordinates of the second point: ");
    scanf("%f%f", &b.x, &b.y);

    printf("Distance (displacement) = %.3f\n", get_length(a, b));

    return 0;
}

float get_length(point a, point b) {
    float dx = b.x - a.x;
    float dy = b.y - a.y;
    return sqrt(dx * dx + dy * dy);
}
```

Explanation

- `point` is a structure that holds `x` and `y` coordinates.
- The distance formula between two points (x_1, y_1) and (x_2, y_2) is $\sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$.
- The `sqrt` function from `math.h` calculates the square root.

7.8 Example: Adding Two Times

Let's create a structure to represent time (hours and minutes) and write a function to add two times.

7.8.1 Example: Time Addition

```c
#include <stdio.h>

typedef struct _time {
    int hour;
    int minutes;
} time;

time findTime(time t1, time t2);
void printTime(time t);

int main() {
    time t1, t2, t3;

    printf("Enter first time (hour minutes): ");
    scanf("%d%d", &t1.hour, &t1.minutes);

    printf("Enter second time (hour minutes): ");
    scanf("%d%d", &t2.hour, &t2.minutes);

    t3 = findTime(t1, t2);

    printf("Total time = ");
    printTime(t3);

    return 0;
}

time findTime(time t1, time t2) {
    time t3;
    int tempmin = t1.minutes + t2.minutes;
```

```
    int temphr;

    // Handle minutes overflow (60 minutes = 1 hour)
    if (tempmin > 59) {
        temphr = tempmin / 60;
        tempmin = tempmin % 60;
    } else {
        temphr = 0;
    }

    t3.minutes = tempmin;
    t3.hour = t1.hour + t2.hour + temphr;

    // Handle 12-hour format
    if (t3.hour > 11) {
        t3.hour = t3.hour % 12;
    }

    return t3;
}

void printTime(time t) {
    printf("%d:%d\n", t.hour, t.minutes);
}
```

Explanation

- `time` is a structure that holds `hour` and `minutes`.
- When adding times, minutes can overflow beyond 60, which means we need to carry over to hours.
- The function returns a `time` structure as the result.

7.9 Common Mistakes with Structures

- Forgetting the semicolon ; after the closing brace of the structure definition.
- Using the wrong member name (spelling mistakes are common).
- Forgetting that strings inside structures are character arrays, so you need `scanf` or `strcpy` to assign them.
- Trying to compare two structures directly using == (this does not work in C).

7.10 Practice Exercises

- Define a structure `Employee` with members: `name`, `id`, `salary`. Write a program that reads details of 5 employees and prints the one with the highest salary.
- Define a structure `Rectangle` with members `length` and `breadth`. Write functions to calculate its area and perimeter, passing the structure to the functions.
- Define a structure `Fraction` with members `numerator` and `denominator`. Write a function to add two fractions and return the result as a structure.

In the next chapter, we will look at pattern printing, where you will learn how to use nested loops to create beautiful output patterns.

Chapter 8

Pattern Printing

8.1 Why Pattern Printing Matters

Pattern printing is one of the best ways to understand how loops work in C. When you print patterns, you learn to control rows and columns using *nested loops*. This skill is essential for working with arrays, matrices, and many other topics in C.

In this chapter, we will learn to create various patterns using stars ($*$), numbers, and spaces.

8.2 The Basic Idea: Nested Loops

Most patterns use two loops working together:

- The *outer loop* controls the number of **rows**.
- The *inner loop* controls the number of characters in each **row**.

After printing all characters in a row, we use `\n` to move to the next line.

8.3 Pattern 1: Right-Angled Star Triangle

The simplest pattern - stars arranged in a right-angled triangle.

8.3.1 Output (for 5 rows)

```
*
* *
* * *
* * * *
* * * * *
```

8.3.2 Code

```c
#include <stdio.h>

int main() {
    int i, j, num;
    scanf("%d", &num);
    for(i=0; i<num; i++) {
        for(j=0; j<=i; j++) {
            printf("* ");
        }
        printf("\n");
    }
}
```

Explanation

- The outer loop runs from 0 to `num-1`, creating `num` rows.
- The inner loop runs from 0 to `i`, so row 0 has 1 star, row 1 has 2 stars, and so on.
- After printing stars for each row, `\n` moves to the next line.

8.4 Pattern 2: Inverted Right-Angled Star Triangle

Now let us flip the previous pattern upside down.

8.4.1 Output (for 5 rows)

```
* * * * *
* * * *
* * *
* *
*
```

8.4.2 Code

```c
#include <stdio.h>

int main() {
    int i, j, num;
    scanf("%d", &num);
    for(i=0; i<num; i++) {
        for(j=0; j<=(num-1)-i; j++) {
            printf("* ");
        }
        printf("\n");
    }
}
```

Explanation

- The outer loop runs from 0 to `num-1`.
- The inner loop runs from 0 to `(num-1)-i`.
- As `i` increases, the number of stars decreases.
- Row 0 has `num` stars, row 1 has `num-1` stars, and so on.

8.5 Pattern 3: Number Pyramid with Spaces

A pattern that uses spaces and numbers to create a centered pyramid.

8.5.1 Output (for 5 rows)

```
    1
   1 2 1
  1 2 3 2 1
 1 2 3 4 3 2 1
1 2 3 4 5 4 3 2 1
```

8.5.2 Code

```c
#include <stdio.h>

int main() {
    int rows;
    short int i, j;
    scanf("%d", &rows);
    for(i=0; i<rows; i++) {
        for(j=0; j<rows-i; j++) {
            printf(" ");
        }
        for(j=1; j<i+1; j++) {
            printf("%d ", j);
        }
        for(j>=1; j--) {
            printf("%d ", j);
        }
        printf("\n");
    }
}
```

Explanation

- Three parts per row: leading spaces, ascending numbers, descending numbers.
- Spaces center the pattern by printing `rows-i` spaces before the numbers.

- The first inner loop prints numbers from 1 up to `i`.
- The second inner loop prints numbers from `i` down to 1.

8.6 Pattern 4: Powers of 2 Pyramid

Each row contains powers of 2, centered with spaces.

8.6.1 Output (for 5 rows)

```
    1
   1 2
  1 2 4
 1 2 4 8
1 2 4 8 16
```

8.6.2 Code

```c
#include <stdio.h>

int main() {
    int rows;
    short int i, j;
    int x=1, k=0;
    scanf("%d", &rows);
    for(i=0; i<rows; i++) {
        for(j=1; j<=rows-i+1; j++) {
            printf(" ");
        }
        for(j=1; j<=i+1; j++) {
            printf("%3d ", x);
        }
        x=x*2;
        printf("\n");
    }
}
```

Explanation

- A variable `x` starts at 1 and is multiplied by 2 after each row.
- Spaces are printed first to center the numbers.
- `%3d` is used to give each number 3 character spaces for alignment.

8.7 Pattern 5: Number Diamond

A diamond shape made of numbers with spaces for alignment.

8.7.1 Output (for 5 rows upper half)

```
    1
   1 2
  1 2 3
 1 2 3 4
1 2 3 4 5
 1 2 3 4
  1 2 3
   1 2
    1
```

8.7.2 Code

```c
#include <stdio.h>

int main() {
    for(int i=1; i<=5; i++) {
        for(int j=1; j<=5-i; j++) {
            printf(" ");
        }
        int j;
        for(j=1; j<=i; j++) {
            printf("%d", j);
```

```
        }
        j-=2;
        while(j>0) {
            printf("%d", j);
            j--;
        }
        printf("\n");
    }
}
```

Explanation

- Spaces are printed first to align the numbers.
- Numbers are printed in ascending order from 1 to `i`.
- Then numbers are printed in descending order from `i-1` down to 1.
- This creates a symmetric diamond-like pattern.

8.8 Pattern 6: Hollow Number Triangle

Only the border of the triangle contains numbers; the inside is empty.

8.8.1 Output (for 5 rows)

```
1
1 2
1   3
1     4
1 2 3 4 5
```

8.8.2 Code

```
#include <stdio.h>
```

```
2
3   int main() {
4       int i, j, num;
5       scanf("%d", &num);
6       for(i=1; i<=num; i++) {
7           for(j=1; j<=i; j++) {
8               if(j==1 || i==j || i==1 || i==num)
9                   printf("%d ", j);
10              else
11                  printf("  ");
12          }
13          printf("\n");
14      }
15  }
```

Explanation

- The `if` condition checks if we are on the border of the triangle.
- Border conditions: first column (`j==1`), diagonal (`i==j`), first row (`i==1`), or last row (`i==num`).
- If on the border, print the number; otherwise, print two spaces.

8.9 Pattern 7: Floyd's Triangle

Floyd's triangle contains consecutive numbers across all rows.

8.9.1 Output (for 5 rows)

```
1
2 3
4 5 6
7 8 9 10
11 12 13 14 15
```

8.9.2 Code

```c
#include <stdio.h>

int main() {
    int n;
    scanf("%d", &n);
    int val = 1;
    for(int i=1; i<=n; i++) {
        for(int j=1; j<=i; j++) {
            printf("%d ", val);
            val++;
        }
        printf("\n");
    }
}
```

Explanation

- A single variable `val` tracks the current number across all rows.
- It starts at 1 and increments after each print.
- The outer loop controls rows, the inner loop controls how many numbers per row.

8.10 Pattern 8: Pascal's Triangle

Pascal's triangle is a famous mathematical pattern where each number is the sum of the two numbers above it.

8.10.1 Output (for 5 rows)

```
    1
  1   1
1   2   1
```

```
  1   3   3   1
1   4   6   4   1
```

8.10.2 Code

```c
#include <stdio.h>

int main() {
    int n;
    scanf("%d", &n);
    for(int i=1; i<=n; i++) {
        for(int j=1; j<=n-i; j++) {
            printf("  ");
        }
        int val = 1;
        for(int j=1; j<=i; j++) {
            printf("%d   ", val);
            val = val * (i-j) / j;
        }
        printf("\n");
    }
}
```

Mathematics Behind It

Each number in Pascal's triangle is a *binomial coefficient*:

- Row n, position $k = \binom{n-1}{k-1}$
- The formula val $=$ val $\times \frac{i-j}{j}$ calculates each coefficient from the previous one.

8.11 Pattern 9: Hollow Diamond

A diamond shape where only the border contains stars.

8.11.1 Output (for 5 rows)

```
*********
**** ****
***   ***
**     **
*       *
*       *
**     **
***   ***
**** ****
*********
```

8.11.2 Code

```c
#include <stdio.h>

int main() {
    int n;
    scanf("%d", &n);

    // Upper half
    for(int i=1; i<=n; i++) {
        for(int j=1; j<=n-i+1; j++) {
            printf("*");
        }
        for(int j=1; j<2*i-1; j++) {
            printf(" ");
        }
        for(int j=1; j<=n-i+1; j++) {
            printf("*");
        }
        printf("\n");
    }

    // Lower half
```

```
    for(int i=1; i<=n; i++) {
        for(int j=1; j<=i; j++) {
            printf("*");
        }
        for(int j=2*n; j>2*i; j--) {
            printf(" ");
        }
        for(int j=1; j<=i; j++) {
            printf("*");
        }
        printf("\n");
    }
}
```

Explanation

- The pattern is split into two parts: upper half and lower half.
- Each row has three parts: left stars, middle spaces, right stars.
- The upper half uses a pyramid logic, the lower half uses inverted pyramid logic.

8.12 Pattern 10: Square of Stars

The simplest pattern - a solid square made of stars.

8.12.1 Output (for 5 rows)

```
* * * * *
* * * * *
* * * * *
* * * * *
* * * * *
```

8.12.2 Code

```c
#include <stdio.h>

int main() {
    int i, j, num;
    scanf("%d", &num);
    for(i=0; i<num; i++) {
        for(j=0; j<num; j++) {
            printf("* ");
        }
        printf("\n");
    }
}
```

Explanation

- Both loops run from 0 to `num-1`.
- Every position in the `num` × `num` grid gets a star.
- This is the simplest nested loop pattern.

8.13 Pattern 11: Chessboard Pattern

A chessboard uses alternating characters in a grid pattern.

8.13.1 Output

```
# # # # # # # #
 # # # # # # # #
# # # # # # # #
 # # # # # # # #
...
```

8.13.2 Code

```c
#include <stdio.h>

int main() {
    short int i=1, j, k=1, x=1;
    while(x<=4) {
        k=1;
        while(k<=2) {
            i=1;
            while(i<=4) {
                j=1;
                while(j<=2) {
                    printf("#");
                    j++;
                }
                j=1;
                while(j<=2) {
                    printf(" ");
                    j++;
                }
                i++;
            }
            printf("\n");
            k++;
        }
        k=1;
        while(k<=2) {
            i=1;
            while(i<=4) {
                j=1;
                while(j<=2) {
                    printf(" ");
                    j++;
                }
                j=1;
                while(j<=2) {
                    printf("#");
```

```
                    j++;
                }
                i++;
            }
            printf("\n");
            k++;
        }
        x++;
    }
}
```

Explanation

- This pattern uses `while` loops instead of `for` loops.
- The board is built row by row, with alternating # and space blocks.
- Each row alternates between starting with # and starting with space.

8.14 Practice Exercises

- Write a program to print a hollow diamond pattern (9 rows total) using asterisks (*) bounded by dashes (-).
- Write a program to print an hourglass pattern with 8 rows using numbers that increase then decrease symmetrically.
- Write a program to print a trapezium pattern (7 rows) where the top row has 3 asterisks, middle has 9, and bottom has 5.

Chapter 9

Sorting Algorithms

9.1 Introduction to Sorting

Sorting is the process of arranging elements of a list or array in a specific order, typically in ascending or descending order. Sorting is a fundamental operation in computer science and is used extensively in various applications such as database management, search algorithms, and data analysis.

This chapter covers five sorting algorithms implemented in C:

1. Bubble Sort
2. Insertion Sort
3. Selection Sort
4. Quick Sort
5. Shell Sort

9.2 Bubble Sort

9.2.1 Concept

Bubble Sort is the simplest sorting algorithm. It repeatedly steps through the list, compares adjacent elements, and swaps them if they are in the wrong order. The process continues until no more swaps are needed, indicating that the list is sorted. The algorithm gets its name from the way larger elements "bubble" to the top (end) of the list.

9.2.2 Algorithm

1. Start from the first element.
2. Compare each pair of adjacent elements.
3. If the first element is greater than the second, swap them.
4. Repeat for each pass through the array.
5. After each pass, the largest unsorted element moves to its correct position at the end.
6. Continue until no swaps occur in a pass.

9.2.3 Time Complexity

- Best Case: $O(n)$ - when the array is already sorted
- Average Case: $O(n^2)$
- Worst Case: $O(n^2)$

9.2.4 Space Complexity

$O(1)$ - Bubble Sort is an in-place sorting algorithm.

9.2.5 Example Code

The bubble_sort.c file implements bubble sort on an array of 6 elements: $\{5, 2, 3, 4, 7, 8\}$.

```c
#include <stdio.h>
int main()
{
    int i, j, temp;
    int arr[6] = {5, 2, 3, 4, 7, 8};
    for(i = 0; i < 6; i++)
    {
        for(j = i+1; j < 6; j++) {
            if(arr[i] > arr[j]) {
                temp = arr[i];
                arr[i] = arr[j];
                arr[j] = temp;
            }
        }
    }
    for(int i=0; i<6; i++)
    {
        printf("%d ", arr[i]);
    }
}
```

9.2.6 Output

The sorted array: `2 3 4 5 7 8`

9.3 Insertion Sort

9.3.1 Concept

Insertion Sort builds the final sorted array one element at a time. It works similarly to the way we sort playing cards in our hands. The array

is virtually split into a sorted and an unsorted part. Elements from the unsorted part are picked and placed at the correct position in the sorted part.

9.3.2 Algorithm

1. Start from the second element (index 1).
2. Store the current element as a key.
3. Compare the key with elements in the sorted portion (to its left).
4. Shift larger elements one position to the right.
5. Insert the key at its correct position.
6. Repeat for all elements.

9.3.3 Time Complexity

- Best Case: $O(n)$ - when the array is already sorted
- Average Case: $O(n^2)$
- Worst Case: $O(n^2)$

9.3.4 Space Complexity

$O(1)$ - Insertion Sort is an in-place sorting algorithm.

9.3.5 Example Code

The insertion_sort.c file implements insertion sort on the array $\{5, 2, 3, 4, 7, 8\}$.

```c
#include <stdio.h>
int main()
{
    int i, j, temp;
    int arr[6] = {5, 2, 3, 4, 7, 8};
```

```
    for(i = 1; i < 6; i++)
    {
        j = i-1;
        temp = arr[i];
        while(j>=0 && temp<arr[j]) {
            arr[j+1] = arr[j];
            j--;
        }
        arr[j+1] = temp;
    }
    for(int i=0; i<6; i++)
    {
        printf("%d ", arr[i]);
    }
}
```

9.3.6 Output

The sorted array: `2 3 4 5 7 8`

9.4 Selection Sort

9.4.1 Concept

Selection Sort divides the array into two parts: sorted and unsorted. It repeatedly finds the minimum element from the unsorted part and places it at the beginning of the sorted part. The algorithm maintains two subarrays: the sorted subarray at the beginning and the remaining unsorted subarray.

9.4.2 Algorithm

1. Find the minimum element in the unsorted portion.

2. Swap it with the element at the current position.

3. Move the boundary between sorted and unsorted portions forward.

4. Repeat until the entire array is sorted.

9.4.3 Time Complexity

- Best Case: $O(n^2)$
- Average Case: $O(n^2)$
- Worst Case: $O(n^2)$

9.4.4 Space Complexity

$O(1)$ - Selection Sort is an in-place sorting algorithm.

9.4.5 Example Code

The selection_sort.c file implements selection sort on the array $\{7, 6, 5, 4, 3, 2, 1\}$.

```c
#include <stdio.h>
int main()
{
    int a[10] = {7, 6, 5, 4, 3, 2, 1};
    int i, j, min, temp;
    for(i=0; i<7; i++)
    {
        min = i;
        for(j=i+1; j<7; j++)
        {
            if(a[j] < a[min]) {
                min = j;
            }
        }
        temp = a[i];
        a[i] = a[min];
        a[min] = temp;
```

```
18      }
19      for(int i=0; i<7; i++) {
20          printf("%d ", a[i]);
21      }
22  }
```

9.4.6 Output

The sorted array: `1 2 3 4 5 6 7`

9.5 Quick Sort

9.5.1 Concept

Quick Sort is a divide-and-conquer algorithm. It works by selecting a "pivot" element from the array and partitioning the other elements into two sub-arrays: elements less than the pivot and elements greater than the pivot. The sub-arrays are then recursively sorted.

9.5.2 Algorithm

1. Choose a pivot element from the array.
2. Partition the array such that elements smaller than the pivot are on the left and larger elements are on the right.
3. Recursively apply the above steps to the left and right sub-arrays.
4. The base case is when the sub-array has one or zero elements.

9.5.3 Time Complexity

- Best Case: $O(n \log n)$
- Average Case: $O(n \log n)$

- Worst Case: $O(n^2)$ - when the pivot is always the smallest or largest element

9.5.4 Space Complexity

$O(\log n)$ - due to the recursion stack.

9.5.5 Example Code

The quick.c file implements Quick Sort with a recursive approach using a midpoint pivot.

```c
#include <stdio.h>

void swap(int *, int, int);
void quick_sort(int *, int, int *);
void qs(int *, int, int, int *);

int main()
{
    int i, a[10], length=10;
    int iter_count = 0;
    printf("Enter 10 integers to be sorted:\n");
    for(i=0; iength; i++)
        scanf("%d", &a[i]);

    printf("\nSorting..... \n");
    quick_sort(a, length, &iter_count);
    printf("\nSorted List: ");
    for(i=0; iength; i++)
        printf("%d ", a[i]);

    printf("\nTotal Iterations=%d\n", iter_count);
    return 0;
}

void swap(int *arr, int i, int j)
```

```
26  {
27      int temp = arr[i];
28      arr[i] = arr[j];
29      arr[j] = temp;
30  }
31
32  void quick_sort(int *arr, int count, int *0)
33  {
34      qs(arr, 0, count-1, 0);
35  }
36
37  void qs(int *arr, int left, int right, int *0)
38  {
39      register int i, j;
40      int pivot, temp;
41      i = left; j = right;
42      pivot = arr[(left+right)/2];
43      do
44      {
45          while((arr[i]<pivot) && (i<right)) i++;
46          while((arr[j]>pivot) && (j>left)) j--;
47          if(i<=j)
48          {
49              swap(arr, i, j);
50              i++, j--;
51              (*0)++;
52          }
53      } while(i<=j);
54      if(left<j) qs(arr, left, j, 0);
55      if(i<right) qs(arr, i, right, 0);
56  }
```

9.6 Shell Sort

9.6.1 Concept

Shell Sort is a generalized version of Insertion Sort. It sorts elements that are far apart first and progressively reduces the gap between elements. By initially sorting elements that are distant, it moves elements closer to their final positions, making the final insertion sort pass more efficient.

9.6.2 Algorithm

1. Choose a gap sequence (e.g., $\{9, 5, 3, 2, 1\}$).
2. For each gap value, perform a modified insertion sort on elements separated by the gap.
3. Reduce the gap and repeat until the gap becomes 1.
4. The final pass with gap 1 is a standard insertion sort.

9.6.3 Time Complexity

- Best Case: $O(n \log n)$
- Average Case: depends on the gap sequence, typically $O(n^{1.5})$
- Worst Case: $O(n^2)$

9.6.4 Space Complexity

$O(1)$ - Shell Sort is an in-place sorting algorithm.

9.6.5 Example Code

The shell.c file implements Shell Sort using the gap sequence $\{9, 5, 3, 2, 1\}$.

```
1 #include <stdio.h>
2
```

```
3  void shell_sort(int*, int, int*);
4
5  int main()
6  {
7      int i, a[10], length=10;
8      int iter_count = 0;
9
10     printf("Enter 10 integers to be sorted:\n");
11     for(i=0; iength; i++)
12         scanf("%d", &a[i]);
13
14     printf("\nSorting..... \n");
15     shell_sort(a, length, &iter_count);
16     printf("\nSorted List: ");
17     for(i=0; iength; i++)
18         printf("%d ", a[i]);
19
20     printf("\nTotal Iterations=%d\n", iter_count);
21     return 0;
22 }
23
24 void shell_sort(int *list, int length, int *O)
25 {
26     register int i, j, k;
27     int temp, gap, a[5];
28     a[0] = 9; a[1] = 5; a[2] = 3; a[3] = 2; a[4] = 1; //setting gaps
29     for(k=0; k<5; k++)  //incrementing gaps
30     {
31         gap = a[k];
32         for(i=gap; iength; i++)  //loop for a gap
33         {
34             temp = list[i];
35             for(j=i-gap; (tempst[j]) && (j>=0); j=j-gap)
36             {
37                 list[j+gap] = list[j];
38                 (*O)++;
39             }
```

```
            list[j+gap] = temp;
        }
    }
}
```

9.7 Comparison of Sorting Algorithms

Algorithm	Best	Average	Worst	Space	Stable
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	No
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	No
Shell Sort	$O(n \log n)$	$O(n^{1.5})$	$O(n^2)$	$O(1)$	No

Table 9.1: Comparison of Sorting Algorithms

9.8 Summary

This chapter presented five sorting algorithms with their implementations in C. The key takeaways are:

- **Bubble Sort** is the simplest but least efficient for large datasets.
- **Insertion Sort** is efficient for small or nearly sorted datasets.
- **Selection Sort** performs well when memory writes are expensive.
- **Quick Sort** is one of the fastest general-purpose sorting algorithms.
- **Shell Sort** improves upon insertion sort by sorting elements at a distance.

For small arrays ($n < 50$), Insertion Sort is often the best choice. For larger arrays, Quick Sort is generally the fastest. Shell Sort offers a good balance between simplicity and performance.

Chapter 10

Miscellaneous Programs and Projects

10.1 Introduction

This concluding chapter presents a collection of diverse C programs that showcase various practical applications of the language. These programs range from real-world simulations like a movie ticket booking system, to mathematical computations like number base conversion and square root calculation, to fundamental data structure implementations and utility programs.

This chapter includes:

1. Book My Show – A Cinema Seat Booking System
2. Decimal to Any Base Conversion
3. Square Root Calculator
4. Fahrenheit to Celsius Converter
5. Leap Year Checker
6. Greatest Common Divisor (GCD)

7. Check Prime Number

8. Stack Implementation

10.2 Book My Show – Cinema Seat Booking System

10.2.1 Overview

The `book_my_show.c` file implements a simple cinema seat booking system called "Clear View Cinema". It demonstrates the use of functions, arrays, switch-case statements, and a menu-driven user interface.

10.2.2 Features

The program provides the following options:

- Display Seats – Shows the current seating arrangement
- Book Ticket – Allows users to book seats
- Cancel Ticket – Allows users to cancel previously booked seats
- Exit – Exits the program

10.2.3 Data Structures Used

- A static array `seats[30]` stores seat information
- `RESERVED = -1` and `FAILURE = 0` are defined as constants
- Vacant seats are stored with their seat number, booked seats are set to 0

10.2.4 Code

```c
#include <stdio.h>

void display_seats();
int book_tickets();
int cancel_tickets();
void initialize();
int check_seat_status();

#define RESERVED -1
#define FAILURE 0
static int seats[30];

int main()
{
    int option=1;
    int result=-1;
    initialize();
    while(option!=4)
    {
        printf("\n");
        printf("Welcome to Clear View Cinema.\n");
        printf("Choose any one of the options below:\n");
        printf("1. Display Seats.\n");
        printf("2. Book Ticket.\n");
        printf("3. Cancel Ticket.\n");
        printf("4. Exit.\n");
        scanf("%d",&option);
        switch(option)
        {
            case 1:
                display_seats();
                break;
            case 2:
            {
                result = book_tickets();
```

```
36                  if(result==FAILURE)
37                      printf("Booking Failed\n");
38                  else
39                      printf("%d tickets booked successfully\n", result);
40                  display_seats();
41                  break;
42              }
43              case 3:
44              {
45                  result = cancel_tickets();
46                  if(result==FAILURE)
47                      printf("Error in cancellation\n");
48                  else {
49                      printf("%d tickets cancelled successfully\n",
                    ↪  result);
50                      display_seats();
51                  }
52                  break;
53              }
54              default:
55                  printf("Thank You\n");
56                  break;
57          }
58      }
59      return 0;
60  }
61
62  void initialize()
63  {
64      int i=0;
65      for(i=0; i<30; i++) {
66          seats[i] = i+1;
67      }
68  }
69
70  void display_seats()
71  {
```

```
72      int i=0;
73      int j=0;
74      printf("SEATING PLAN\n");
75      for(i=0; i<30; i++)
76      {
77          if(i==10 || i==20)
78              printf("\n");
79          printf("%2d ", seats[i]);
80      }
81      printf("\n");
82  }
83
84  int book_tickets()
85  {
86      int num, seat_no, count=0;
87      printf("Please enter the number of tickets to be booked\n");
88      scanf("%d", &num);
89      if(num<1 || num>30)
90          return FAILURE;
91      else {
92          count=num;
93          printf("Please enter the seat numbers one by one\n");
94          do {
95              scanf("%d", &seat_no);
96              if(seats[seat_no-1]!=0) {
97                  seats[seat_no-1] = 0;
98                  --count;
99              }
100             else if(seat_no <1 || seat_no >30)
101                 printf("Invalid Seat number, please enter 1-30\n");
102             else {
103                 printf("Seat already booked. choose vacant seat\n");
104             }
105         } while(count!=0);
106         return num;
107     }
108 }
```

```
109
110 int cancel_tickets()
111 {
112     int num, seat_no, count=0;
113     printf("Please enter the number of tickets to cancel\n");
114     scanf("%d", &num);
115     if(num<1 || num>30 || check_seat_status()<num)
116         return FAILURE;
117     else {
118         count=num;
119         printf("Please enter the seat numbers one by one\n");
120         do {
121             scanf("%d", &seat_no);
122             if(seats[seat_no-1]==0) {
123                 seats[seat_no-1] = seat_no;
124                 --count;
125             }
126             else {
127                 printf("Seat already vacant. choose booked seat\n");
128             }
129         } while(count!=0);
130         return num;
131     }
132 }
133
134 int check_seat_status()
135 {
136     int i=0, count=0;
137     for(i=0; i<30; i++) {
138         if(seats[i]==0)
139             count++;
140     }
141     return count;
142 }
```

Explanation

- The `initialize()` function sets up the initial seat status with numbers 1 to 30
- `display_seats()` prints the current seating arrangement in a formatted way
- `book_tickets()` allows the user to book multiple seats at once
- `cancel_tickets()` cancels previously booked tickets
- `check_seat_status()` counts the number of booked seats

Key Learning Points

- Use of multiple functions to organize code
- Static arrays for persistent data
- Switch-case for menu-driven programs
- Input validation for seat numbers
- Tracking state changes (booked vs. vacant seats)

10.3 Decimal to Any Base Conversion

10.3.1 Overview

The `decimal_to_r_base.c` file demonstrates how to convert a decimal number to any base between 2 and 16. This is useful for understanding how computers represent numbers internally.

10.3.2 Algorithm

1. Divide the decimal number by the target base
2. Store the remainder

3. Continue dividing until the quotient becomes 0
4. Read the remainders in reverse order
5. For bases above 10, use letters A-F for digits 10-15

10.3.3 Code

```c
#include<stdio.h>

char *decimal2anybase(int, int);

int main()
{
    int num10 = 875;
    int base = 8;
    char *numr;
    numr = decimal2anybase(num10, base);
    printf("Converted Number: %s\n", numr);
    return 0;
}

char *decimal2anybase(int num10, int base)
{
    int rem=1;
    int i=0, j=0;
    int temp[20];
    static char str[20];

    while(num10>0)
    {
        if(base>16)
            return "INVALID INPUT";
        rem = num10%base;
        num10 = num10/base;
        temp[i] = rem;
        i++;
    }
```

```
    i--;
    while(i>=0)
    {
        if(temp[i]<10)
            str[j] = (char)(temp[i]+48);
        else if(temp[i]==10)
            str[j]='A';
        else if(temp[i]==11)
            str[j]='B';
        else if(temp[i]==12)
            str[j]='C';
        else if(temp[i]==13)
            str[j]='D';
        else if(temp[i]==14)
            str[j]='E';
        else if(temp[i]==15)
            str[j]='F';
        j++; i--;
    }
    str[j] = '\0';
    if(str!=NULL)
        return str;
    else
        return "ERROR";
}
```

Output

For input `num10 = 875` and `base = 8`, the output is:

```
Converted Number: 1557
```

Explanation

- The function repeatedly divides the number by the target base and collects remainders

- Remainders are stored in reverse order in the `temp` array
- Digits 0-9 are converted to ASCII characters using `(char)(digit+48)`
- Digits 10-15 are converted to letters A-F for hexadecimal representation

Key Learning Points

- Using the modulo operator for digit extraction
- Converting digits to characters using ASCII values
- Handling bases up to hexadecimal (base 16)
- Using static arrays to return strings from functions

10.4 Square Root Calculator

10.4.1 Overview

The `square_root.c` file implements a square root calculator without using the built-in `sqrt()` function. It uses a digit-by-digit calculation method similar to long division.

10.4.2 Algorithm

1. Find the largest digit whose square is less than or equal to the number
2. This becomes the integer part of the square root
3. For the fractional part, bring down pairs of zeros
4. Find the next digit using the trial divisor method
5. Repeat for the desired accuracy

10.4.3 Code

```c
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

int main()
{
    int i=0, j=0;
    int accuracy=6;
    int num = 21;
    int quotient, remainder, divisor, frac;
    quotient = remainder = divisor = frac = 0;
    char *ans = malloc(sizeof(char)*10);

    while(i*i<=num) {
        i++;
    }
    i--;
    divisor = quotient = i;
    remainder = num-(i*i);
    sprintf(ans, "%d", quotient);

    ans = strcat(ans, ".");
    char *temp = malloc(sizeof(char)*10);
    frac = quotient;
    i=0; j=0;

    while(accuracy>0)
    {
        divisor = (divisor+frac)*10;
        remainder = remainder*100;
        while((divisor+j)*j<=remainder) {
            j++;
        }
        j--;
        frac = remainder/(divisor+j);
```

```
        remainder = remainder%(divisor+j);
        sprintf(temp, "%d", frac);
        ans = strcat(ans, temp);
        accuracy--;
    }
    printf("%s", ans);
}
```

Output

For `num = 21` with `accuracy = 6`, the output is approximately:

```
4.582575
```

Explanation

- The program first finds the integer part of the square root
- Then it adds a decimal point and calculates fractional digits
- The divisor is updated in each iteration to find the next digit
- Memory is dynamically allocated for the result string

Key Learning Points

- Dynamic memory allocation using `malloc()`
- String manipulation with `sprintf()` and `strcat()`
- Manual square root calculation algorithm
- Controlling decimal precision

10.5 Fahrenheit to Celsius Converter

10.5.1 Overview

The `fahrenheit_celsius.c` file is a simple temperature conversion program that demonstrates floating-point arithmetic in C.

10.5.2 Code

```c
#include<stdio.h>

int main()
{
    float fahrenheit=0.0, celsius=0.0;

    printf("Enter a Fahrenheit temperature: ");
    scanf("%f", &fahrenheit);

    celsius = fahrenheit - 32.0f;
    celsius = celsius * 5.0f / 9.0f;

    printf("That equals %f degrees Celsius.", celsius);
}
```

Output

For input `fahrenheit = 98.6`, the output is:

```
That equals 37.000000 degrees Celsius.
```

Explanation

- The program takes Fahrenheit temperature as input
- Subtracts 32 from the Fahrenheit value
- Multiplies by 5 and divides by 9 to get Celsius
- The result is printed with floating-point precision

Key Learning Points

- Using `float` data type for decimal values
- Floating-point arithmetic operations
- The conversion formula: $C = (F - 32) \times \frac{5}{9}$

10.6 Leap Year Checker

10.6.1 Overview

The `leap_year.c` file checks whether a given year is a leap year using if-else conditions.

10.6.2 Rules for Leap Year

A year is a leap year if:

1. It is divisible by 400, OR
2. It is divisible by 4 but NOT divisible by 100

10.6.3 Code

```c
#include <stdio.h>

int main()
{
    int year=2400;

    if(year%400==0)
    {
        printf("The year is leap");
    }
    else if(year%100==0)
    {
        printf("The year is not leap");
```

```
    }
    else if(year%4==0)
    {
        printf("The year is leap");
    }
    else
    {
        printf("The year is not leap");
    }
}
```

Output

For `year = 2400`, the output is:

```
The year is leap
```

Explanation

- The program checks divisibility by 400 first (top priority)
- Then checks divisibility by 100 (which excludes most century years)
- Then checks divisibility by 4 (regular leap year rule)
- If none of these apply, it is not a leap year

Key Learning Points

- Using nested if-else statements
- Modulo operator for divisibility checks
- Understanding the leap year algorithm

10.7 Greatest Common Divisor (GCD)

10.7.1 Overview

The `GCD.c` file implements the Euclidean algorithm to find the greatest common divisor of two numbers.

10.7.2 Algorithm

The Euclidean algorithm works by repeatedly replacing the larger number with the remainder of dividing the larger by the smaller, until the remainder is 0. The last non-zero remainder is the GCD.

10.7.3 Code

```c
#include <stdio.h>

int GCD(int x, int y)
{
    int temp;
    while(y!=0)
    {
        temp = y;
        y = x % y;
        x = temp;
    }
    return x;
}

int main()
{
    int num1 = 100;
    int num2 = 120;
    printf("%d\n", GCD(num1, num2));
    return 0;
}
```

Output

For `num1 = 100` and `num2 = 120`, the output is:

```
20
```

Explanation

- The function is called with two numbers
- In each iteration, the larger number is replaced with the remainder
- The loop continues until the remainder becomes 0
- The value of `x` at that point is the GCD

Example Walkthrough

For `GCD(100, 120)`:

1. `x = 100, y = 120`: remainder is 100
2. `x = 120, y = 100`: remainder is 20
3. `x = 100, y = 20`: remainder is 0
4. `x = 20, y = 0`: return 20

Key Learning Points

- The Euclidean algorithm for GCD
- Using a while loop for iterative computation
- Function with parameters and return value
- Swapping values using a temporary variable

10.8 Check Prime Number

10.8.1 Overview

The `check_prime.c` file determines whether a given number is prime or not.

10.8.2 Code

```c
#include <stdio.h>

int main()
{
    int N, i=2;
    printf("Enter a number: ");
    scanf("%d",&N);

    if(N<2)
    {
        printf("The number is invalid\n");
        return 1;
    }

    while(i!=N/2)
    {
        if(N%i==0)
        {
            printf("The number is not prime");
            break;
        }
        i++;
    }

    if(i>=N/2)
        printf("The number is prime");

    return 0;
```

```
29 }
```

Output

For input `N = 17`, the output is:

```
Enter a number: 17
The number is prime
```

For input `N = 18`, the output is:

```
Enter a number: 18
The number is not prime
```

Explanation

- The program first checks if the number is less than 2 (invalid)
- It then iterates from 2 to N/2 to find factors
- If any factor is found, the number is not prime
- If no factors are found, the number is prime

Optimization Note

We only need to check up to `N/2` because any factor larger than `N/2` would have a corresponding factor smaller than `N/2`.

Key Learning Points

- Using a while loop for prime checking
- Input validation for numbers less than 2
- Early exit using `break` when a factor is found
- Loop optimization techniques

10.9 Stack Implementation

10.9.1 Overview

The `stack.c` file implements a basic stack data structure, which is a Last-In-First-Out (LIFO) data structure. In a stack, the last element inserted is the first one to be removed.

10.9.2 Operations Supported

- `push()` – Insert an element onto the stack
- `pop()` – Remove the top element from the stack
- `peek()` – View the top element without removing it
- `display()` – Show all elements in the stack
- `isFull()` – Check if the stack is full
- `isEmpty()` – Check if the stack is empty

10.9.3 Code

```c
#include <stdio.h>

#define MAX 5

int stack[MAX];
int top = -1;

void push(int value)
{
    if(top == MAX - 1)
    {
        printf("Stack is full!\n");
        return;
    }
    stack[++top] = value;
```

```
16      printf("%d pushed to stack\n", value);
17  }
18
19  int pop()
20  {
21      if(top == -1)
22      {
23          printf("Stack is empty!\n");
24          return -1;
25      }
26      int value = stack[top--];
27      printf("%d popped from stack\n", value);
28      return value;
29  }
30
31  int peek()
32  {
33      if(top == -1)
34      {
35          printf("Stack is empty!\n");
36          return -1;
37      }
38      return stack[top];
39  }
40
41  void display()
42  {
43      if(top == -1)
44      {
45          printf("Stack is empty!\n");
46          return;
47      }
48      printf("Stack elements: ");
49      for(int i = 0; i <= top; i++)
50      {
51          printf("%d ", stack[i]);
52      }
```

```
    printf("\n");
}

int main()
{
    int choice, value;

    while(1)
    {
        printf("\n1. Push\n2. Pop\n3. Peek\n4. Display\n5. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch(choice)
        {
            case 1:
                printf("Enter value to push: ");
                scanf("%d", &value);
                push(value);
                break;
            case 2:
                pop();
                break;
            case 3:
                printf("Top element: %d\n", peek());
                break;
            case 4:
                display();
                break;
            case 5:
                printf("Exiting...\n");
                return 0;
            default:
                printf("Invalid choice!\n");
        }
    }
```

```
89 }
```

Output

Sample execution:

```
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter value to push: 10
10 pushed to stack
...
Enter your choice: 4
Stack elements: 10 20 30
```

Explanation

- `top` variable tracks the position of the top element (-1 means empty)
- `push()` adds an element and increments `top`
- `pop()` removes an element and decrements `top`
- `peek()` returns the top element without modifying the stack
- `display()` iterates from 0 to `top` to show all elements

Key Learning Points

- Implementing a LIFO (Last-In-First-Out) data structure
- Array usage to store stack elements
- Menu-driven program design

- Boundary checking for overflow and underflow
- Multiple operations on a single data structure
- Practical application of arrays and functions

Advantages of Stack

- Simple to implement using arrays
- Efficient for function calls and recursion
- Used in expression evaluation and parsing
- Memory management in function call stacks

This concludes the comprehensive collection of C programs presented in this book. Each program demonstrates fundamental programming concepts and can serve as a foundation for more advanced applications.